

PENJELASAN ALGORITMA DEEP LEARNING

Prediksi Curah Hujan India dengan LSTM

1. LSTM (Long Short-Term Memory)

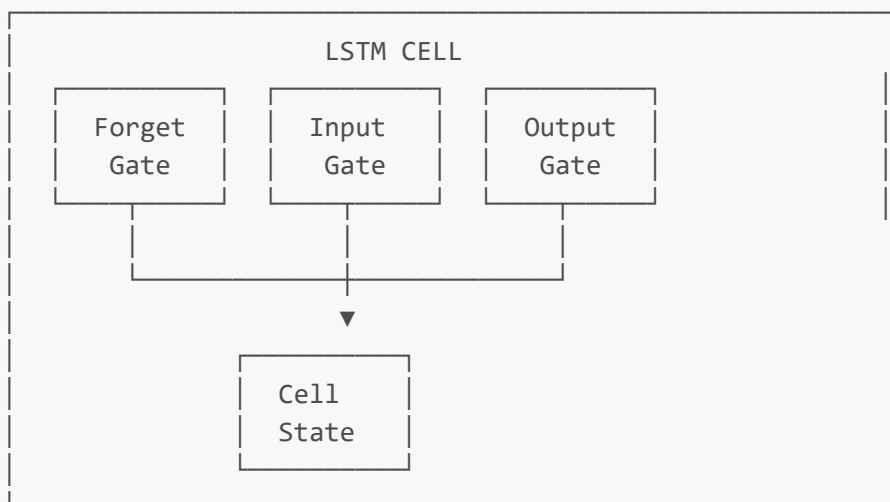
Apa itu LSTM?

LSTM adalah jenis **Recurrent Neural Network (RNN)** yang mampu mengingat informasi dalam jangka panjang. Berbeda dengan RNN biasa yang "lupa" informasi lama, LSTM punya mekanisme khusus untuk menyimpan dan mengatur memori.

Mengapa LSTM untuk Prediksi Curah Hujan?

- Data curah hujan adalah **time series** (data berurutan waktu)
- Curah hujan hari ini dipengaruhi oleh pola hari-hari sebelumnya
- LSTM bisa "mengingat" pola musiman dan tren jangka panjang

Arsitektur LSTM - 4 Komponen Utama



A. Forget Gate (Gerbang Lupa)

Fungsi: Memutuskan informasi mana yang harus "dilupakan" dari memori sebelumnya.

Rumus:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Cara Baca:

- f_t = output forget gate (nilai 0-1)
- σ = fungsi sigmoid (menghasilkan nilai 0-1)
- W_f = bobot (weight) untuk forget gate
- h_{t-1} = output tersembunyi dari waktu sebelumnya
- x_t = input saat ini
- b_f = bias

Analogi: Seperti otak yang memutuskan "apakah cuaca minggu lalu masih relevan untuk prediksi besok?"

- Nilai mendekati 0 = lupakan
- Nilai mendekati 1 = ingat

B. Input Gate (Gerbang Input)

Fungsi: Memutuskan informasi baru mana yang akan disimpan ke memori.

Rumus (2 bagian):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \leftarrow \text{seberapa penting info baru?}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \leftarrow \text{kandidat nilai baru}$$

Cara Baca:

- i_t = seberapa banyak info baru yang masuk (0-1)
- $\tilde{C}_t$ = kandidat nilai baru (-1 sampai 1)
- $\tanh$ = fungsi yang menghasilkan nilai -1 sampai 1

Analogi: Seperti memutuskan "curah hujan hari ini penting untuk disimpan, dan nilainya tinggi/rendah"

C. Cell State Update (Pembaruan Memori)

Fungsi: Menggabungkan memori lama yang dipertahankan dengan informasi baru.

Rumus:

$$C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t$$

Cara Baca:

- C_t = cell state (memori) baru
- $f_t \times C_{t-1}$ = memori lama yang dipertahankan
- $i_t \times \tilde{C}_t$ = informasi baru yang ditambahkan

Analogi: Seperti buku catatan yang diperbarui - hapus yang tidak relevan, tambah yang baru.

D. Output Gate (Gerbang Output)

Fungsi: Memutuskan bagian mana dari memori yang akan menjadi output.

Rumus:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \times \tanh(C_t)$$

Cara Baca:

- o_t = seberapa banyak memori yang dikeluarkan (0-1)
- h_t = hidden state (output) yang diteruskan

Implementasi dalam Proyek

```
LSTM(64, return_sequences=True, input_shape=(30, 1)) # Layer 1
LSTM(32, return_sequences=False) # Layer 2
```

Penjelasan Parameter:

Parameter	Nilai	Arti
<code>units</code>	64, 32	Jumlah neuron LSTM
<code>return_sequences</code>	True/False	True = keluarkan semua timestep, False = hanya timestep terakhir
<code>input_shape</code>	(30, 1)	30 hari data, 1 fitur (curah hujan)

2. DROPOUT (Regularisasi)

Apa itu Dropout?

Teknik untuk **mencegah overfitting** dengan secara acak "mematikan" sebagian neuron saat training.

Cara Kerja

```
Training: [●] [○] [●] [●] [○] [●] ← beberapa neuron dimatikan (○)
Testing:  [●] [●] [●] [●] [●] [●] ← semua neuron aktif
```

Rumus

```
Training: y = f(W × (x ⊙ mask)) / (1 - dropout_rate)
Testing:  y = f(W × x)
```

Cara Baca:

- `mask` = vektor acak berisi 0 dan 1
- $\odot$ = perkalian element-wise
- `dropout_rate` = persentase neuron yang dimatikan

Implementasi

```
Dropout(0.2) # 20% neuron dimatikan secara acak
```

Efek Dropout Rate:

Rate	Efek
0.0	Tidak ada dropout (terbaik untuk dataset besar)
0.2	Ringan, cocok untuk baseline
0.5	Agresif, bisa menurunkan performa

3. DENSE LAYER (Fully Connected)

Apa itu Dense Layer?

Layer di mana **setiap neuron terhubung ke semua neuron** di layer sebelumnya.

Rumus

$$y = f(W \times x + b)$$

Cara Baca:

- `W` = matriks bobot (weight)
- `x` = input
- `b` = bias
- `f` = fungsi aktivasi

Implementasi

```
Dense(16, activation='relu') # Hidden layer dengan 16 neuron  
Dense(1)                    # Output layer (1 nilai prediksi)
```

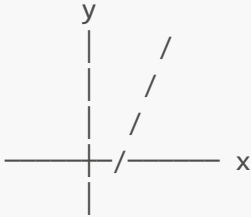
4. FUNGSI AKTIVASI

A. ReLU (Rectified Linear Unit)

Rumus:

$$\text{ReLU}(x) = \max(0, x)$$

Grafik:



Keunggulan:

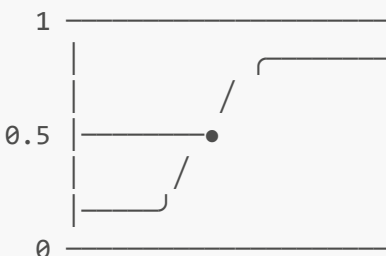
- Komputasi cepat
- Mengatasi vanishing gradient
- Menghasilkan sparse activation

B. Sigmoid

Rumus:

$$\sigma(x) = 1 / (1 + e^{(-x)})$$

Grafik:

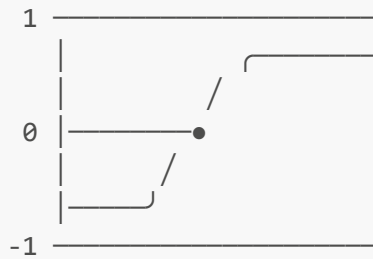


Kegunaan: Digunakan di gate LSTM karena outputnya 0-1 (seperti probabilitas)

C. Tanh (Hyperbolic Tangent)

Rumus:

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

Grafik:

Kegunaan: Digunakan untuk candidate values di LSTM karena outputnya -1 sampai 1

5. OPTIMIZER - ADAM

Apa itu Adam?

Adam = Adaptive Moment Estimation Optimizer yang menggabungkan keunggulan Momentum dan RMSprop.

Rumus Lengkap

- | | |
|----------------------|---|
| 1. Hitung gradient: | $g_t = \nabla L(\theta_t)$ |
| 2. Update momentum: | $m_t = \beta_1 \times m_{t-1} + (1-\beta_1) \times g_t$ |
| 3. Update velocity: | $v_t = \beta_2 \times v_{t-1} + (1-\beta_2) \times g_t^2$ |
| 4. Koreksi bias: | $\hat{m}_t = m_t / (1-\beta_1^t)$ |
| | $\hat{v}_t = v_t / (1-\beta_2^t)$ |
| 5. Update parameter: | $\theta_{t+1} = \theta_t - \alpha \times \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ |

Cara Baca:

- g_t = gradient (arah penurunan error)
- m_t = momentum (rata-rata gradient)
- v_t = velocity (rata-rata kuadrat gradient)
- α = learning rate (0.001 default)
- β_1 = 0.9 (decay rate momentum)
- β_2 = 0.999 (decay rate velocity)
- ϵ = 1e-8 (mencegah pembagian dengan nol)

Implementasi

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

Perbandingan Optimizer

Optimizer	Kelebihan	Kekurangan
Adam	Adaptif, konvergen cepat	Bisa overshoot
SGD	Sederhana, stabil	Lambat konvergen
RMSprop	Baik untuk RNN	Kurang adaptif

6. LOSS FUNCTION - MSE

Mean Squared Error (MSE)

Rumus:

$$\text{MSE} = (1/n) \times \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

Cara Baca:

- Hitung selisih antara nilai aktual dan prediksi
- Kuadratkan selisih tersebut
- Rata-ratakan semua kuadrat selisih

Contoh:

Aktual: [10, 20, 30]
Prediksi: [12, 18, 32]
Selisih: [-2, 2, -2]
Kuadrat: [4, 4, 4]
MSE = (4+4+4)/3 = 4

Karakteristik:

- Memberikan penalti besar untuk error besar
- Sensitif terhadap outlier
- Selalu positif

7. EVALUATION METRICS

A. RMSE (Root Mean Squared Error)

Rumus:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{(1/n) \times \sum (y - \hat{y})^2}$$

Interpretasi: Satuan sama dengan data asli (mm curah hujan)

B. MAE (Mean Absolute Error)

Rumus:

$$MAE = (1/n) \times \sum |y - \hat{y}|$$

Interpretasi: Rata-rata kesalahan absolut, lebih robust terhadap outlier

C. R² Score (Coefficient of Determination)

Rumus:

$$R^2 = 1 - (SS_{res} / SS_{tot})$$

$$SS_{res} = \sum (y - \hat{y})^2 \quad \leftarrow \text{Sum of Squared Residuals}$$

$$SS_{tot} = \sum (y - \bar{y})^2 \quad \leftarrow \text{Total Sum of Squares}$$

Interpretasi:

Nilai R ²	Arti
1.0	Prediksi sempurna
0.97	97% variasi data dijelaskan model
0.0	Model sama buruknya dengan rata-rata
< 0	Model lebih buruk dari rata-rata

8. PREPROCESSING

A. Min-Max Scaling

Rumus:

$$x_{scaled} = (x - x_{min}) / (x_{max} - x_{min})$$

Contoh:

```
Data asli: [100, 200, 300, 400, 500]
x_min = 100, x_max = 500
```



```
Scaled:      [0.0, 0.25, 0.5, 0.75, 1.0]
```

Mengapa perlu scaling?

- Neural network bekerja lebih baik dengan nilai 0-1
- Mempercepat konvergensi training
- Mencegah dominasi fitur dengan nilai besar

Implementasi

```
scaler = MinMaxScaler(feature_range=(0, 1))  
scaled_data = scaler.fit_transform(data)
```

B. Sliding Window (Sequence Creation)

Konsep: Mengubah time series menjadi pasangan input-output untuk supervised learning.

Ilustrasi (window = 3):

```
Data: [10, 20, 30, 40, 50, 60]  
  
Sequence 1: Input=[10,20,30] → Output=40  
Sequence 2: Input=[20,30,40] → Output=50  
Sequence 3: Input=[30,40,50] → Output=60
```

Implementasi:

```
seq_length = 30 # Gunakan 30 hari untuk prediksi hari ke-31  
  
for i in range(len(data) - seq_length):  
    X.append(data[i:i+seq_length]) # 30 hari input  
    y.append(data[i+seq_length])   # 1 hari output
```

9. CALLBACKS

A. Early Stopping

Fungsi: Menghentikan training jika tidak ada improvement.

```
EarlyStopping(  
    monitor='val_loss', # Pantau validation loss
```

```
    patience=10,                # Tunggu 10 epoch tanpa improvement
    restore_best_weights=True # Kembalikan bobot terbaik
)
```

Ilustrasi:

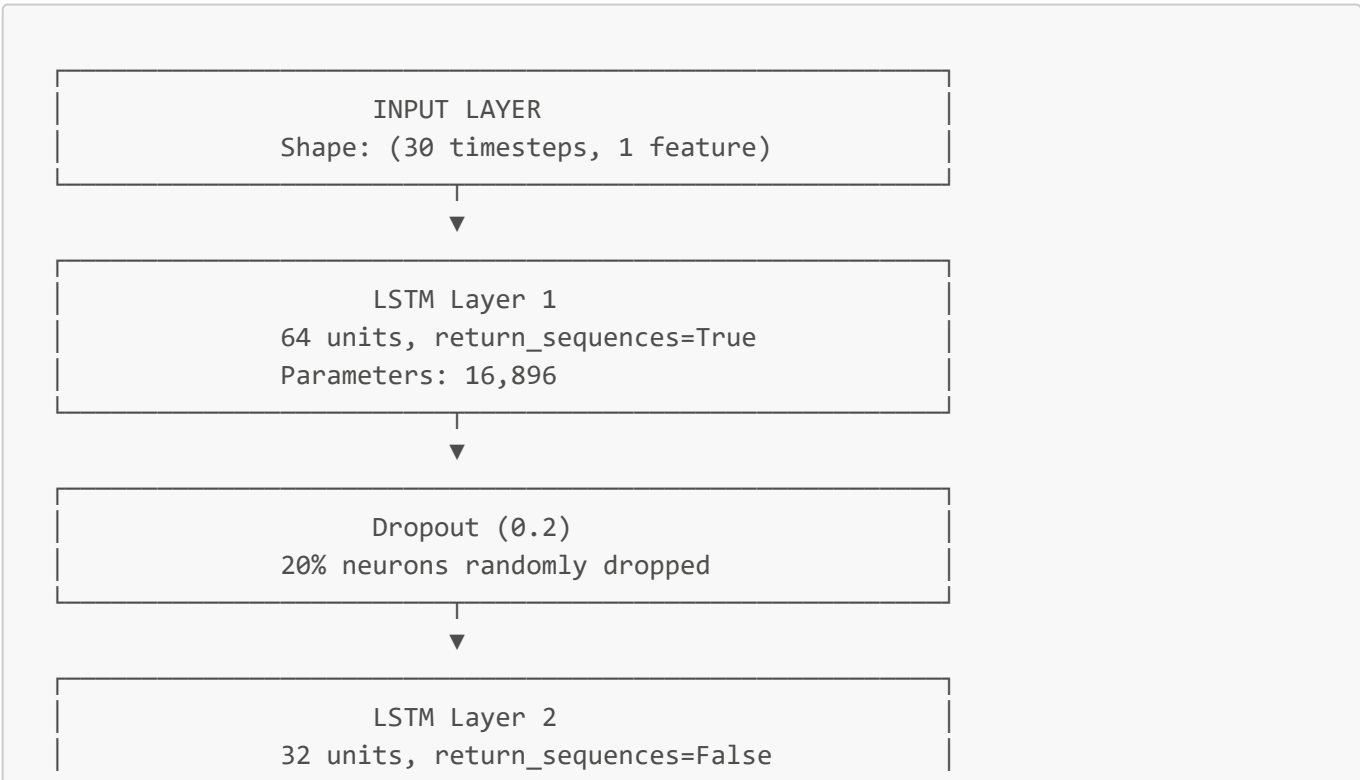
Epoch	Val_Loss	Status
1	0.0021	✓ Improving
2	0.0019	✓ Improving
3	0.0020	✗ No improvement (1/10)
4	0.0018	✓ Improving (reset counter)
...		
14	0.0025	✗ No improvement (10/10) → STOP!

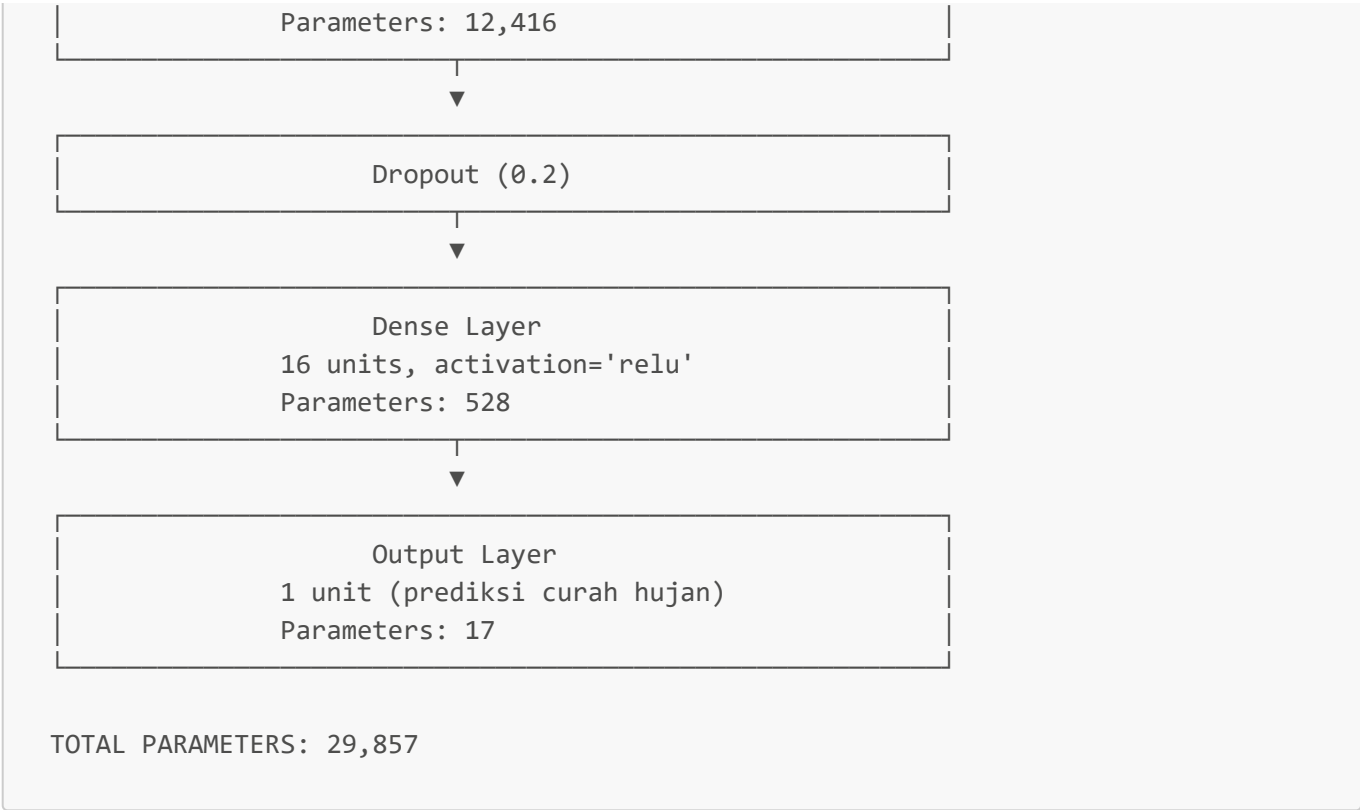
B. Model Checkpoint

Fungsi: Menyimpan model terbaik selama training.

```
ModelCheckpoint(
    'best_model.h5',           # Nama file
    monitor='val_loss',        # Pantau validation loss
    save_best_only=True        # Simpan hanya yang terbaik
)
```

10. ARSITEKTUR MODEL LENGKAP





11. HYPERPARAMETER YANG DIGUNAKAN

Hyperparameter	Nilai	Penjelasan
Sequence Length	30	Gunakan 30 hari untuk prediksi
LSTM Units	64, 32	Jumlah neuron per layer
Dense Units	16	Neuron di hidden dense layer
Dropout Rate	0.2	20% neuron dimatikan
Learning Rate	0.001	Kecepatan belajar (default Adam)
Batch Size	32	Jumlah sampel per update
Epochs	100	Maksimum iterasi training
Patience	10	Early stopping patience

12. HASIL MODEL

Metrik	Nilai	Interpretasi
MSE	0.00093	Error kuadrat sangat kecil
RMSE	0.0305	Error ~0.03 (skala 0-1)
MAE	0.0107	Rata-rata error absolut kecil
R ² Score	0.9746	97.46% variasi data dijelaskan

13. RINGKASAN ALUR KERJA

1. LOAD DATA
 - └ Baca CSV curah hujan India 1901-2015
2. PREPROCESSING
 - └ Konversi bulanan → harian
 - └ Min-Max Scaling (0-1)
 - └ Buat sequences (sliding window 30 hari)
3. SPLIT DATA
 - └ 80% training, 20% testing
4. BUILD MODEL
 - └ LSTM → Dropout → LSTM → Dropout → Dense → Output
5. COMPILE
 - └ Optimizer: Adam, Loss: MSE
6. TRAIN
 - └ Dengan Early Stopping & Model Checkpoint
7. EVALUATE
 - └ Hitung MSE, RMSE, MAE, R^2
8. PREDICT
 - └ Prediksi curah hujan hari berikutnya

Dokumen ini menjelaskan setiap komponen algoritma deep learning yang digunakan dalam proyek prediksi curah hujan India menggunakan LSTM, dengan rumus matematika dan penjelasan yang mudah dipahami.